Linear Ops Playbook

Cómo AltruPets organiza su workspace

Para Ingeniería -- Operaciones -- Todos los equipos
Marzo 2026 -- v2.0

Qué vamos a cubrir

#	Tema	Detalle
01	**Arquitectura del Workspace**	Team ALT, 8 proyectos, y cómo se conecta todo
02	**Sistema de Labels**	La taxonomía de 31 labels y cómo usarla
03	**Guía de Creación de Issues**	El template bilingüe para humanos + agentes IA
04	**OpenSpec Workflow**	Spec library, changes, y el flujo proposal-to-code
05	**Desarrollo AI-First**	Estrategias de agentes, context budgets, paralelización
06	**Gobernanza y Rituales**	Triage, auditorías semanales, y reglas del juego

01

Arquitectura del Workspace

1 team -- 8 proyectos -- Un workspace unificado

Team AltruPets (ALT)

Un solo team con 8 proyectos que cubren todo el stack:

Proyecto	Foco
Backend API	NestJS, GraphQL, TypeORM, PostgreSQL
Mobile App	Flutter, Riverpod, Clean Architecture
Agent AI	agentic-core sidecar, LangGraph, FalkorDB
Infrastructure & DevOps	K8s, Terraform, ArgoCD, CI/CD
Web -- B2G	Vue 3 panel gubernamental
	Migración a Flutter Web/Jaspr en evaluación (ALT-40)
Web -- CRUD Superusers	Vue 3 panel admin
	Migración a Flutter Web/Jaspr en evaluación (ALT-41)
Diseño y Control de Calidad	UX/UI, QA
Operaciones	Admin, legal, partnerships

Ciclo de Vida de un Issue

Cada issue fluye por estos estados. Linear los trackea nativamente -- no se necesitan labels.

Backlog --> **Todo** --> **In Progress** --> **In Review** --> **Done**

Y también: **Canceled** (documentar por qué) + **Duplicate** (linkear original)

Niveles de Prioridad (built-in)

Nivel	Cuándo usarlo
Urgent	Producción caída, breach de SLA. Máx 5 - 7 en todo el workspace.
High	Importante para el ciclo actual. Impacto en revenue o estrategia.
Medium	Debe hacerse pero no rompe nada si se atrasa un ciclo.
Low	Nice to have. Llenar gaps de sprint.

02

El Sistema de Labels

31 labels organizados en grupos y categorías

Principios de Diseño de Labels

- **Labels agregan lo que Linear no tiene**
  - Linear ya tiene Priority, Status, Estimates, Projects, Cycles
  - Labels agregan: Type, Component, Business Impact, y AI Strategy
- **Grupos = mutuamente exclusivos**
  - Label Groups fuerzan UN label por grupo por issue
  - Un issue es UN tipo, UN tamaño, UNA estrategia
- **Sin grupo = combinables**
  - Labels de Component pueden superponerse
  - Ejemplo: Frontend + Security + Testing en un mismo issue
- **Cada label tiene descripción**
  - Las descripciones definen CUÁNDO y CUÁNDO NO usar cada label
- **A nivel de workspace**
  - Los 31 labels son visibles para todos los proyectos
- **El color codifica la categoría**
  - Cada grupo tiene una familia de colores para identificación visual

Los 3 Grupos de Labels

Type (exclusivo, requerido)

Label	Descripción
Bug	Algo está roto. Crashes, errores, violaciones de spec.
Feature	Capacidad nueva que no existe. Endpoint, página, evento.
Improvement	Mejora a funcionalidad existente. UX, perf, refactor.
Chore	Mantenimiento. Sin cambio visible. Deps, CI/CD, docs.
Spike	Investigación acotada. Output = conocimiento. ADR, PoC.
Design	UI/UX o creativo. Mockups, design system, branding.

Size (exclusivo, tokens IA)

Label	Tokens	Tiempo	Ejemplo
XS	<50K	~30 min	Un archivo, typo, config
S	50-100K	~2-4 hrs	Componente, hook, migración
M	100-200K	~1-2 días	Frontend + backend + tests
L	200-500K	~3-5 días	Cross-layer, arquitectura
XL	500K+	Descomponer	Epic, múltiples PRs

Strategy (exclusivo, agente IA)

Label	Cuándo usarlo
Solo	Reqs claros, un agente end-to-end
Team	Múltiples agentes en paralelo
Worktree	Aislamiento. Cambios riesgosos
Explore	Investigar codebase primero
Human	Requiere decisión humana
Review	Solo auditoría, sin código

Labels Sin Grupo -- Combinables

Issues pueden tener múltiples labels de estas categorías.

Component (8 labels)

Label	Área
Frontend	Flutter mobile o web UI
Backend	NestJS, API, GraphQL
Database	Schema, migraciones, ORM
Security	Auth, secrets, encryption
Performance	Optimización, caching
Infra	K8s, Terraform, CI/CD
Testing	Unit, integration, e2e
Web Quality	A11y, SEO, Core Web Vitals

Impact (3 labels)

Label	Cuándo
Critical Path	Bloquea otro trabajo o un lanzamiento
Revenue	Ligado a cliente pagador o deal
Grant	Financiado por grants, deadlines externos

Flags (3 labels)

Label	Cuándo
Blocked	Esperando dependencia
Quick Win	<2 horas, alto impacto
Epic	Contenedor padre con sub-issues

Total: 31 labels = 3 Grupos (17) + Component (8) + Impact (3) + Flags (3)

Labeling en Práctica -- 3 Ejemplos

Issue	Type	Size	Strategy	Component	Impact
"Agregar verificación de email al onboarding"	Feature	S	Solo	Backend, Security	
"Fix webhook de subsidio retornando 500 en aprobación"	Bug	M	Explore	Backend	Critical Path, Revenue
"Asegurar contrato municipal de San José"	Feature		Human		Revenue

03

Escribiendo Buenos Issues

El template bilingüe -- diseñado para humanos Y agentes IA

El Template Bilingüe

Cada issue debe ser legible y valioso para humanos que orquestan Y agentes que ejecutan.

> *La Capa Humana no es solo para humanos -- previene alucinaciones del agente al proveer conocimiento de dominio y edge cases.*

Capa Humana

- **User Story**
  - Como [rol], quiero [X] para que [Y]
- **Background / Por qué**
  - 2-3 párrafos, lenguaje llano
- **Analogía**
  - Comparar con algo familiar
- **UX / Referencia Visual**
  - Screenshots, links Figma
- **Pitfalls Conocidos**
  - Edge cases, data legacy

Capa Agente

- **Objetivo**
  - 1-2 oraciones, outcome técnico
- **Context Files**
  - Paths exactos con descripción
- **Acceptance Criteria**
  - Checkboxes, testeables
- **Technical Constraints**
  - Patrones a seguir
- **Verification Commands**
  - Comandos bash exactos
- **Context Budget + Agent Strategy**

Por Qué Issues Ricos Ganan

> *La economía de specs ricos vs. alucinaciones de agentes*

Issue rico de 50 líneas:
~500 tokens extra (0.25% de ventana de 200K) + 10 min humano = agente ejecuta correctamente al primer intento

Issue vago de 2 líneas:
Agente alucina paths + inventa reglas de negocio = rework completo, ~50K tokens quemados, múltiples ciclos de review

> *El enriquecimiento de issues es la actividad humana de mayor apalancamiento. 10 minutos agregando contexto ahorran 30+ minutos de debugging.*

04

OpenSpec Workflow

Spec library, changes, y el flujo de proposal a código

Qué es OpenSpec

OpenSpec agrega una **capa de especificación** al monorepo que los agentes IA consultan antes de escribir código.

Spec Library (estado actual)

Documentación viva de cómo funciona el sistema HOY. Organizada por capability/workflow:

- `platform-overview/spec.md`
- `rescue-pipeline/spec.md`
- `vet-subsidy/spec.md`
- `adoption/spec.md`
- `financial/spec.md`
- `abuse-reports/spec.md`
- `architecture/spec.md`

Changes (propuestas)

Cada feature nueva sigue el flujo:

1. `proposal.md` -- Qué y por qué
2. `design.md` -- Cómo (arquitectura, APIs)
3. `tasks.md` -- Tareas con checkboxes

Al implementar y archivar, las deltas se fusionan de vuelta en la spec library.

El Flujo OpenSpec

Comando	Acción
`/opsx:propose "descripción"`	Crea proposal + design + tasks
`/opsx:apply nombre-del-change`	Lee artefactos, implementa task por task
`/opsx:archive nombre-del-change`	Fusiona deltas en spec library

Integración con Linear

Cada change se vincula a un issue de Linear (ALT-XX). El issue contiene la referencia al directorio del change. Para implementar: `/opsx:apply` o `/make-no-mistakes ALT-XX`

Spec Library -- 7 specs, 4,923 líneas

Spec	Líneas	Contenido clave
platform-overview	320	Roles, revenue, roadmap
vet-subsidy	639	Saga, 44 rules
financial	766	PCI DSS, KYC

05

Desarrollo AI-First

Estrategias de agentes, paralelización, y la regla de 200K

La Regla de 200K

Ningún issue debería requerir más contexto que una ventana de Claude (200K tokens).

Cuándo Dividir

- Toca más de 5 archivos
- Cruza 2+ capas (frontend + backend + DB)
- Abarca 2+ dominios (rescue + payments)
- Etiquetado como XL (500K+ tokens)

Cómo Dividir

- Dividir por capa (frontend / backend / DB)
- Dividir por boundary de dominio
- Research spike + Implementation + Review
- Usar issue padre (Epic) para trackear el set

3 Mecanismos de Paralelización

Mecanismo	Mejor para	Analogía	Costo
Subagents	Research rápido, tareas focalizadas	"Enviar un asistente a buscar algo"	1x tokens
Git Worktrees	Sesiones paralelas en diferentes branches	"Escritorios separados en la misma oficina"	1x por sesión
Agent Teams	Trabajo multi-parte complejo	"Firma consultora auto-organizada"	3-4x tokens

Mapeo Size a Mecanismo

Size	→	Mecanismo
XS/S	→	Solo (sin paralelización)
M (un componente)	→	Solo o Subagents para research
M (múltiples componentes)	→	Agent Teams (2 teammates)
L	→	Agent Teams (2-3) o Worktree si riesgoso
XL	→	Descomponer primero, Agent Teams por sub-issue

El Flujo de Ingeniería

1. **Issue creado** -- Usar template bilingüe. Setear Type + Size + Strategy.
2. **Humano enriquece** -- Agregar pitfalls, context files, acceptance criteria. 10 min = actividad de mayor ROI.
3. **Agente lee** -- Lee issue COMPLETO + OpenSpec change si existe.
4. **Agente implementa** -- Sigue Technical Constraints. Ejecuta Verification Commands.
5. **Agente crea PR** -- Mueve issue a In Review. Spawns review subagent.
6. **Humano revisa PR** -- Calidad, lógica de negocio, edge cases. Aprobar o pedir cambios.
7. **Merge --> Done** -- PR merged a main. Issue auto-moves a Done. Post-merge monitoring.

3 Checkpoints Humanos: Issue enrichment (#2) -- PR review (#6) -- Post-merge monitoring (#7)

06

Gobernanza y Rituales

Triage semanal, gobernanza de labels, y reglas del camino

Triage y Rituales de Gobernanza

Triage Semanal (Viernes)

- Revisar top 10 issues sin asignar
- Aplicar template bilingüe a issues de alta prioridad
- Asignar a ciclo de sprint
- Re-verificar conteo de Urgent
  - Máx 5-7 en todo el workspace

Auditoría de Labels (Mensual)

- Ejecutar `/linear-projects-setup audit`
- Verificar que no se crearon labels no autorizados
- Confirmar que todos los issues abiertos tienen label Type
- Archivar labels en desuso

Poda de Backlog (Trimestral)

- Cancelar issues stale >90 días sin actividad
- Re-priorizar issues Urgent (matar inflación)
- Descomponer cualquier issue XL restante

Reglas del Camino

- Solo workspace admins pueden crear nuevos labels
- Todo issue necesita un **Type** -- sin excepciones
- Usar `blockedBy` / `blocks` para dependencias
- Todo issue pertenece a un **proyecto**
- **Conventional commits** obligatorios: `feat:`, `fix:`, `docs:`, `chore:`
- **No emojis** ni diagramas ASCII en el repo -- usar Mermaid
- **PR reviewers automáticos**: Greptile + CodeRabbit + Graphite
- `linear-setup.json` es la fuente de verdad para configuración del proyecto

Conventional Commits

Todos los commits siguen el estándar Conventional Commits. Enforcement vía pre-commit hook.

Formato

Elemento	Separador	Elemento	Resultado

Tipos permitidos

Tipo	Cuándo usarlo	Ejemplo
`feat`	Nueva funcionalidad	`feat: add subsidy request workflow`
`fix`	Corrección de bug	`fix: resolve 500 on webhook approval`
`docs`	Solo documentación	`docs: expand financial spec`
`style`	Formato, sin cambio lógico	`style: convert ASCII diagrams to Mermaid`
`refactor`	Reestructuración sin cambio funcional	`refactor: extract matching service`
`perf`	Mejora de rendimiento	`perf: add Redis cache to proximity queries`
`test`	Agregar o corregir tests	`test: add unit tests for adoption flow`

Breaking Changes

Agregar `!` después del tipo o `BREAKING CHANGE:` en el footer:

```
feat!: change subsidy API response format
```

Scopes comunes en AltruPets

- **`backend`**
  - Cambios en `apps/backend/`
- **`mobile`**
  - Cambios en `apps/mobile/`
- **`infra`**
  - Cambios en `k8s/`, `infrastructure/`
- **`web`**
  - Cambios en `apps/web/`
- **`openspec`**
  - Cambios en `openspec/`
- **`agent`**
  - Cambios en `apps/agent-sidecar/`

Ejemplos completos

Custom Slash Commands -- OpenSpec

Planificación

`/opsx:propose "descripción"`

Crea un nuevo change con los 3 artefactos (proposal, design, tasks) en un solo paso.

`/opsx:explore`

Modo exploración -- pensar ideas, investigar problemas, clarificar requerimientos. Sin implementar código.

`/opsx:archive nombre`

Archiva un change completado. Fusiona deltas de vuelta en la spec library.

Implementación

`/opsx:apply nombre`

Lee los artefactos del change e implementa las tasks una por una. Marca checkboxes conforme avanza.

`/make-no-mistakes ALT-XX`

Protocolo de ejecución disciplinada: worktree, Greptile review loop, CI verification, merge limpio.

`/spec-recommend T0-4`

Analiza un task del SRD gap audit y genera un OpenSpec change con brief de implementación.

Custom Slash Commands -- Operaciones y Testing

Gestión de Workspace

`/linear-projects-setup`

Bootstrap del workspace: labels, proyectos, milestones. Modos: `audit`, `labels`, `projects`, `sync`.

`/linear-projects-setup audit`

Dry run que muestra el diff entre estado actual y taxonomía. Sin mutaciones.

`/summarize`

Resume la sesión actual con decisiones tomadas, archivos modificados, y próximos pasos.

`/commit`

Crea un commit siguiendo conventional commits con análisis de cambios staged.

Testing y Review

`/e2e-test-builder ruta/al/doc.md`

Genera un `test-suite.json` compatible con TestSprite desde documentación de use cases.

`/e2e-test-runner`

Ejecuta tests E2E desde `test-suite.json`. Selecciona el runner óptimo (Playwright, Chrome DevTools MCP, Flutter).

`/review-pr`

Review comprehensivo de PR usando agentes especializados.

`/code-review`

Code review de un pull request con análisis de calidad.

Quick Reference

Cheat Sheet

Cheat Sheet

Dimensión	Labels	Regla
Type	Bug, Feature, Improvement, Chore, Spike, Design	Uno por issue, requerido
Size	XS, S, M, L, XL	Uno por issue, mapea a tokens
Strategy	Solo, Team, Worktree, Explore, Human, Review	Uno por issue, guía al agente
Component	Frontend, Backend, DB, Security, Perf, Infra, Testing, Web Quality	Combinables
Impact	Critical Path, Revenue, Grant	Combinables
Flags	Blocked, Quick Win, Epic	Combinables

Comandos Clave

Comando	Función
`/opsx:propose`	Crear change con artefactos
`/opsx:apply`	Implementar tasks de un change
`/make-no-mistakes ALT-XX`	Ejecutar con disciplina completa
`/spec-recommend T0-4`	Analizar y generar change desde SRD
`/linear-projects-setup audit`	Verificar estado del workspace

A Construir.

Un workspace. 8 proyectos. 31 labels.

Issues ricos. Ejecución AI-first. Calidad human-in-the-loop.

Todo issue tiene Type. Sin excepciones.
Issues bilingües -- 10 min de enriquecimiento ahorra 30 min de rework.
Size + Strategy guían la ejecución del agente IA.
Triage de viernes mantiene el backlog saludable.